

Anotações — Aula 5: Métricas, Validação Cruzada e Tunagem de Hiperparâmetros

Disciplina: Machine Learning — Pós-Graduação Ciência de Dados e Big Data / Estatística

Professor: Murilo Afonso Robiati Bigoto (PUC Minas)

Datasets: UCI Heart Disease (notebook principal) · Breast Cancer (notebook de tuning)

Notebooks: aula_ml_classificacao_v2.ipynb · Exemplo_aula5_tuning_rand.ipynb

Frase-chave: “O conjunto de teste é tocado uma única vez, ao final do processo. Qualquer decisão de modelagem — seleção de modelo, tuning — usa apenas treino e validação cruzada.”

Onde estamos no Ciclo de Vida (CRISP-DM)

Entendimento do Negócio	Entendimento dos Dados	Preparação dos Dados	Modelagem (treino)	Avaliação aqui	Implantação (deploy)
-------------------------	------------------------	----------------------	--------------------	----------------	----------------------

Esta aula conecta a **etapa de Modelagem** à **etapa de Avaliação**: como validar corretamente, tunar hiperparâmetros sem vaziar informação, e interpretar as métricas certas para cada problema.

Pipeline Completo da Aula

Dataset

1. EDA (distribuição de classes, correlações, features discriminativas)
2. Pré-processamento
 - Remove duplicatas
 - Imputa missings (mediana)
 - Clipping de outliers (percentis 1%–99%)

3. Split estratificado — 70% treino / 30% teste teste SELADO aqui
4. Tuning com Optuna/RandomizedSearchCV + Validação Cruzada (5-fold)
Critério de seleção: maior AUC média nos folds
5. Seleção do melhor modelo via ranking de AUC (CV)
6. Avaliação final único momento em que o teste é tocado
Métricas: Acurácia, Precisão, Recall, F1, AUC-ROC

PARTE 1 — Divisão Treino e Teste

O conceito fundamental

O objetivo do ML não é acertar dados que já vimos, mas ter **poder preditivo sobre dados nunca vistos**. Por isso separamos rigorosamente treino e teste.

Conceito	O que é
Underfitting	Modelo simples demais — não aprende nem o treino
Overfitting	Modelo “decora” o treino, incluindo ruído
Generalização	O “ponto doce” — performa bem nos dois conjuntos

Estrutura Hold-out

```
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.3,    # 30% para teste
    stratify=y,      # mantém proporção das classes
    random_state=42
)
```

```
print(f'Treino : {len(X_train)} amostras | target=1: {y_train.mean():.2%}')
print(f'Teste : {len(X_test)} amostras | target=1: {y_test.mean():.2%}')
```

*Regra de ouro: O conjunto de teste é **selado** no momento do split e só é aberto uma única vez, para o reporte final. Usar o teste para tomar qualquer decisão de modelagem é **data leakage** e invalida os resultados.*

Técnicas de Divisão

A. Estratificação (stratify=y)

Essencial para classificação. Garante que a proporção de classes (ex: 10% fraude) seja idêntica no treino e no teste. Sem isso, você pode treinar sem nenhum exemplo de uma classe minoritária.

B. Divisão Temporal (para séries temporais)

Split aleatório é um **erro grave** em dados temporais — constitui *look-ahead bias* (o modelo “vê o futuro”).

```
# ERRADO para séries temporais
X_train, X_test = train_test_split(X, test_size=0.2) # mistura passado e futuro!

# CERTO
n = len(df)
X_train = df.iloc[:int(0.8 * n)] # passado
X_test = df.iloc[int(0.8 * n):] # futuro
```

C. Conjunto de Validação (projetos complexos)

Em projetos mais robustos usa-se **três conjuntos**: Treino Validação Teste. A validação serve para tuning; o teste final é tocado uma única vez.

Analogia: O conjunto de teste é como a prova final do semestre. O conjunto de validação são os simulados que você faz para ajustar seu estudo. Se você treinar exatamente para a prova final (testá-la antes), a nota não reflete o seu conhecimento real.

PARTE 2 — Validação Cruzada (Cross-Validation)

Por que usar?

Um único split treino/teste pode ser “sortudo ou azarado”. A validação cruzada repete a avaliação múltiplas vezes e retorna uma **estimativa mais estável e confiável** da performance.

***Analogia:** Em vez de avaliar um aluno com uma única prova, você o avalia com 5 provas diferentes ao longo do semestre. A média das 5 notas é uma estimativa muito mais justa do seu conhecimento do que uma única prova num dia bom ou ruim.*

Tipos de Cross-Validation

Tipo	Como funciona	Quando usar
k-Fold	Divide em k partes; treina em k-1, testa na restante	Caso geral
Stratified k-Fold	k-Fold mantendo proporção de classes em cada fold	Classificação (padrão)
Leave-One-Out (LOOCV)	Cada observação é um fold	Datasets muito pequenos
Time Series Split	Treino sempre precede o teste no tempo	Séries temporais
Group k-Fold	Garante que o mesmo grupo não aparece em treino e teste	Dados com grupos (clientes, pacientes)

Código — Stratified k-Fold

```
from sklearn.model_selection import StratifiedKFold, cross_val_score
from sklearn.ensemble import RandomForestClassifier

cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)

model = RandomForestClassifier(n_estimators=100, random_state=42)

# cross_val_score retorna a métrica para cada fold
auc_scores = cross_val_score(
    model, X_train, y_train,
    cv=cv,
    scoring='roc_auc' # métrica de avaliação em cada fold
```

```
)
```

```
print(f'AUC por fold: {auc_scores.round(4)}')  
print(f'AUC média : {auc_scores.mean():.4f} ± {auc_scores.std():.4f}')
```

Aplicações práticas da CV:

1. **Seleção de modelos** — comparar algoritmos de forma justa
2. **Tuning de hiperparâmetros** — garantir que a configuração não decora um único split
3. **Datasets pequenos** — aproveitar ao máximo cada amostra disponível

PARTE 3 — Tunagem de Hiperparâmetros

Parâmetros vs Hiperparâmetros

	Parâmetros	Hiperparâmetros
Quem define	O algoritmo, durante o treino	Você, antes do treino
Exemplos	Pesos de uma regressão, thresholds de uma árvore	<code>max_depth</code> , <code>learning_rate</code> , <code>n_estimators</code>
Como ajustar	Automático (<code>.fit()</code>)	Busca manual, grid, random ou bayesiana

A. Grid Search — Busca Exaustiva

Testa **todas as combinações** possíveis do espaço de hiperparâmetros definido.

```
from sklearn.model_selection import GridSearchCV  
from sklearn.ensemble import RandomForestClassifier  
  
param_grid = {  
    'n_estimators' : [100, 200, 500],  
    'max_depth' : [None, 10, 20, 30],  
    'min_samples_split': [2, 5, 10],  
    'criterion' : ['gini', 'entropy']  
}
```

```
# 3 × 4 × 3 × 2 = 72 combinações × 5 folds = 360 treinos!
busca = GridSearchCV(
    RandomForestClassifier(random_state=42),
    param_grid,
    cv=5,
    scoring='roc_auc',
    n_jobs=-1
)
busca.fit(X_train, y_train)

print(f'Melhores parâmetros: {busca.best_params_}')
print(f'Melhor AUC (CV) : {busca.best_score_:.4f}')
```

***Explosão combinatória:** adicionar mais um parâmetro com 3 valores multiplica o custo por 3. Com 5 parâmetros de 5 valores cada $5^5 \times \text{folds} = 15.625$ treinos. Impraticável em produção.*

B. Random Search — Busca Aleatória

Em vez de testar tudo, **amostra combinações aleatórias** dentro de distribuições definidas.

```
from sklearn.model_selection import RandomizedSearchCV
import numpy as np

param_distributions = {
    'n_estimators' : np.arange(50, 500, 25),
    'max_depth' : [None, 10, 20, 30],
    'min_samples_split': np.arange(2, 20, 2),
    'min_samples_leaf': np.arange(1, 10),
    'max_features' : ['sqrt', 'log2']
}

busca_rand = RandomizedSearchCV(
    RandomForestClassifier(random_state=42),
    param_distributions,
    n_iter=50,      # quantas combinações sortear
    cv=5,
    scoring='roc_auc',
```

```

    random_state=42,
    n_jobs=-1
)
busca_rand.fit(X_train, y_train)

print(f'Melhores parâmetros: {busca_rand.best_params_}')
print(f'Melhor AUC (CV) : {busca_rand.best_score_:.4f}')

```

Por que o Random Search funciona? Estudos mostram que nem todos os hiperparâmetros têm igual importância. O Grid Search desperdiça iterações variando parâmetros secundários. O Random Search explora melhor o espaço dos parâmetros realmente importantes com uma fração do custo.

C. Otimização Bayesiana com Optuna — A Busca Inteligente

Diferente das anteriores, a busca bayesiana é **sequencial e adaptativa**: aprende com os resultados anteriores para decidir qual combinação testar a seguir.

```

import optuna
optuna.logging.set_verbosity(optuna.logging.WARNING)

from sklearn.model_selection import StratifiedKFold, cross_val_score
from sklearn.ensemble import RandomForestClassifier

SEED = 42
cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=SEED)

# Função objetivo: define o espaço de busca e retorna a métrica
def objective_rf(trial):
    params = {
        'n_estimators' : trial.suggest_int('n_estimators', 50, 500),
        'max_depth'     : trial.suggest_int('max_depth', 2, 20),
        'min_samples_split': trial.suggest_int('min_samples_split', 2, 20),
        'min_samples_leaf': trial.suggest_int('min_samples_leaf', 1, 10),
        'max_features'   : trial.suggest_categorical('max_features', ['sqrt', 'log2', None]),
        'class_weight'   : trial.suggest_categorical('class_weight', ['balanced', None]),
        'random_state'   : SEED,
    }
    model = RandomForestClassifier(**params)
    auc_scores = cross_val_score(model, X_train, y_train, cv=cv, scoring='roc_auc')

```

```

return auc_scores.mean() # Optuna maximiza este valor

# Criar e executar o estudo
study_rf = optuna.create_study(direction='maximize')
study_rf.optimize(objective_rf, n_trials=30, show_progress_bar=True)

print(f'Melhor AUC : {study_rf.best_value:.4f}')
print(f'Melhores params: {study_rf.best_params}')

```

Configuração do Optuna para XGBoost:

```

from xgboost import XGBClassifier

def objective_xgb(trial):
    params = {
        'n_estimators' : trial.suggest_int('n_estimators', 50, 500),
        'max_depth'     : trial.suggest_int('max_depth', 2, 10),
        'learning_rate' : trial.suggest_float('learning_rate', 0.01, 0.3, log=True),
        'subsample'     : trial.suggest_float('subsample', 0.5, 1.0),
        'colsample_bytree' : trial.suggest_float('colsample_bytree', 0.5, 1.0),
        'gamma'         : trial.suggest_float('gamma', 0.0, 5.0),
        'reg_alpha'      : trial.suggest_float('reg_alpha', 1e-4, 10.0, log=True), # L1
        'reg_lambda'     : trial.suggest_float('reg_lambda', 1e-4, 10.0, log=True), # L2
        'eval_metric'    : 'logloss',
        'random_state'   : SEED,
        'verbosity'     : 0
    }
    model = XGBClassifier(**params)
    return cross_val_score(model, X_train, y_train, cv=cv, scoring='roc_auc').mean()

```

Método	Custo	Qualidade	Quando usar
Grid Search	Alto	Ótimo (exaustivo)	Poucos hiperparâmetros, espaço pequeno
Random Search	Médio	Bom	Prototipagem rápida, espaços grandes
Optuna (Bayesiana)	Baixo	Excelente	Produção, espaços grandes, qualidade máxima

O fluxo correto de tuning (sem data leakage)

Dataset completo

X_train (70%) entra no GridSearchCV/Optuna

Validação Cruzada Interna (k-Fold)
fold 1: treina em 2-5, valida em 1
fold 2: treina em 1,3-5, valida em 2
...
Escolhe melhores hiperparâmetros

X_test (30%) tocado UMA ÚNICA VEZ com o modelo final

Erro clássico: tunar hiperparâmetros usando o conjunto de teste. Se você faz isso, o modelo “vicia” no teste e a métrica final será otimista demais — uma mentira estatística.

PARTE 4 — Métricas de Performance

Métricas de Classificação

A Matriz de Confusão

Predito

Neg Pos

Real Neg TN FP FP = Falso Positivo (Alarme falso)

Pos FN TP FN = Falso Negativo (Miss)

Sigla	Nome	O que é
TP	True Positive	Modelo disse positivo era positivo
TN	True Negative	Modelo disse negativo era negativo
FP	False Positive	Modelo disse positivo era negativo (alarme falso)
FN	False Negative	Modelo disse negativo era positivo (miss perigoso)

As 4 métricas principais

```
from sklearn.metrics import (accuracy_score, precision_score,
                             recall_score, f1_score,
                             classification_report)

y_pred = model.predict(X_test)

print(f'Acurácia : {accuracy_score(y_test, y_pred):.4f}')
print(f'Precisão : {precision_score(y_test, y_pred):.4f}')
print(f'Recall : {recall_score(y_test, y_pred):.4f}')
print(f'F1-Score : {f1_score(y_test, y_pred):.4f}')
print()
print(classification_report(y_test, y_pred))
```

Métrica	Fórmula	Quando priorizar
Acurácia	$(TP+TN) / \text{total}$	Classes balanceadas, custo simétrico dos erros
Precisão	$TP / (TP+FP)$	Custo alto de falso positivo (ex: spam filter)
Recall	$TP / (TP+FN)$	Custo alto de falso negativo (ex: diagnóstico de câncer)
F1-Score	$2 \times (P \times R) / (P + R)$	Classes desbalanceadas, equilíbrio P/R

Analogia Precisão vs Recall:

- **Precisão** = “Dos alarmes de incêndio que soaram, quantos eram incêndios reais?” (evitar alarmes falsos)
- **Recall** = “De todos os incêndios reais, quantos o alarme detectou?” (não perder nenhum fogo)
Em diagnóstico médico, Recall é crítico — é pior perder uma doença do que dar um falso positivo que será investigado depois.

AUC-ROC

Mede a capacidade do modelo de **ordenar** corretamente as probabilidades — dar score maior para positivos do que para negativos — independentemente do threshold de corte.

```
from sklearn.metrics import roc_auc_score, roc_curve
import matplotlib.pyplot as plt
```

```
# Precisa das probabilidades, não das classes
y_proba = model.predict_proba(X_test)[:, 1] # P(classe=1)

auc = roc_auc_score(y_test, y_proba)
fpr, tpr, thresholds = roc_curve(y_test, y_proba)

plt.figure(figsize=(8, 6))
plt.plot(fpr, tpr, lw=2, label=f'AUC = {auc:.4f}')
plt.plot([0, 1], [0, 1], 'k--', label='Modelo aleatório (AUC=0.5)')
plt.xlabel('Taxa de Falsos Positivos (FPR)')
plt.ylabel('Taxa de Verdadeiros Positivos (TPR = Recall)')
plt.title('Curva ROC')
plt.legend()
plt.show()
```

AUC	Interpretação
1.0	Modelo perfeito
0.9 – 1.0	Excelente
0.8 – 0.9	Bom
0.7 – 0.8	Razoável
0.5	Modelo aleatório (inútil)
< 0.5	Pior que o acaso

Por que AUC é preferida em datasets desbalanceados? A acurácia pode ser enganosa: em fraude com 99% de negativos, um modelo que diz “não é fraude” para tudo acerta 99% — mas tem AUC = 0.5. A AUC mede o poder real de discriminação entre classes.

Métricas de Regressão

```
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
import numpy as np

y_pred_reg = model.predict(X_test)
```

```

mae = mean_absolute_error(y_test, y_pred_reg)
rmse = np.sqrt(mean_squared_error(y_test, y_pred_reg))
r2 = r2_score(y_test, y_pred_reg)

print(f'MAE : {mae:.4f}') # erro médio na unidade original
print(f'RMSE : {rmse:.4f}') # penaliza erros grandes
print(f'R² : {r2:.4f}') # % da variância explicada

```

Métrica	Característica	Exemplo de uso
MAE	Linear, não penaliza outliers	Previsão de tempo de entrega (iFood)
RMSE	Penaliza erros grandes fortemente	Temperatura de turbina de avião
R²	% da variância explicada (0–1)	Qualidade geral de ajuste do modelo

*Quando usar RMSE vs MAE: Se errar por R\$100 uma vez é **catastrófico** (turbina de avião, dose de remédio), use RMSE — ele “grita” com erros grandes. Se errar por R\$10 dez vezes e R\$100 uma vez tiverem o mesmo impacto, use MAE — ele trata todos os erros igualmente.*

PARTE 5 — Pipeline Completo: Dataset Heart Disease

Dataset UCI Heart Disease

303 pacientes avaliados clinicamente para doença coronariana.

Target: target (1 = doença presente, 0 = ausente)

```

from sklearn.datasets import fetch_openml

dataset = fetch_openml(name='heart-statlog', version=1, as_frame=True)
df = dataset.frame.copy()

# Target como binário
df['target'] = (df['class'] == 'present').astype(int)
df.drop(columns=['class'], inplace=True)

print(f'Shape: {df.shape}')
print(f'Proporção target=1: {df["target"].mean():.2%}')

```

Pré-processamento

```
# 1. Duplicatas
df.drop_duplicates(inplace=True)

# 2. Missings (mediana — robusta a outliers)
for col in df.columns:
    if df[col].isnull().any():
        df[col].fillna(df[col].median(), inplace=True)

# 3. Clipping de outliers extremos (percentil 1% – 99%)
continuous_cols = [c for c in df.columns if c != 'target' and df[c].nunique() > 10]
for col in continuous_cols:
    p01 = df[col].quantile(0.01)
    p99 = df[col].quantile(0.99)
    df[col] = df[col].clip(lower=p01, upper=p99)
```

Tuning com Optuna + Avaliação Final

```
import optuna
optuna.logging.set_verbosity(optuna.logging.WARNING)

N_TRIALS = 20 # 30-50 em produção; 20 é adequado para aula
CV_FOLDS = 5
cv = StratifiedKFold(n_splits=CV_FOLDS, shuffle=True, random_state=SEED)

results = {}

# --- Roda o tuning para cada modelo ---
for nome, (study, objective) in {
    'XGBoost' : (optuna.create_study(direction='maximize'), objective_xgb),
    'LightGBM' : (optuna.create_study(direction='maximize'), objective_lgb),
    'Random Forest' : (optuna.create_study(direction='maximize'), objective_rf),
    'Árvore de Decisão' : (optuna.create_study(direction='maximize'), objective_dt),
    'Regressão Logística': (optuna.create_study(direction='maximize'), objective_lr),
}.items():
    study.optimize(objective, n_trials=N_TRIALS, show_progress_bar=False)
    results[nome] = {
        'best_auc' : study.best_value,
```

```

        'best_params': study.best_params
    }

# --- Ranking ---
comparison_df = pd.DataFrame([
    {'Modelo': k, 'AUC CV': round(v['best_auc'], 4)}
    for k, v in results.items()
]).sort_values('AUC CV', ascending=False).reset_index(drop=True)

print(comparison_df)

```

Avaliação Final (abrindo o teste uma única vez)

```

# Treina o melhor modelo com os hiperparâmetros encontrados
best_params = results['Random Forest']['best_params']
final_model = RandomForestClassifier(**best_params, random_state=SEED)
final_model.fit(X_train, y_train)

# --- Predições ---
y_pred = final_model.predict(X_test)
y_pred_proba = final_model.predict_proba(X_test)[:, 1]

# --- Métricas ---
print(f'Acurácia : {accuracy_score(y_test, y_pred):.4f}')
print(f'Precisão : {precision_score(y_test, y_pred):.4f}')
print(f'Recall : {recall_score(y_test, y_pred):.4f}')
print(f'F1-Score : {f1_score(y_test, y_pred):.4f}')
print(f'AUC-ROC : {roc_auc_score(y_test, y_pred_proba):.4f}')
print()
print(classification_report(y_test, y_pred))

```

Pontos-Chave para Fixar

1. **O teste é selado no início** — qualquer uso antes da avaliação final é data leakage
2. **Estratificação** (`stratify=y`) é obrigatória em classificação com classes desbalanceadas
3. **Validação Cruzada** é mais confiável do que um único split — use para selecionar modelos e tunar hiperparâmetros

4. **Grid Search** é exaustivo e sofre de explosão combinatória; **Random Search** é mais eficiente; **Optuna** é o estado da arte
 5. **AUC-ROC** é a métrica principal para classificação com desbalanceamento — acurácia pode enganar
 6. **Precisão vs Recall**: escolha conforme o custo relativo de FP e FN no seu problema de negócio
 7. **RMSE vs MAE**: use RMSE quando erros grandes são catastróficos; MAE quando todos os erros têm peso igual
 8. `predict_proba()[, 1]` retorna a probabilidade da classe positiva — necessário para AUC e threshold adjustment
 9. **TPE (Optuna)** aprende com tentativas anteriores para sugerir combinações melhores — muito mais eficiente que busca aleatória
 10. O **gráfico de convergência do Optuna** mostra se o tuning convergiu ou se mais trials seriam úteis
-

Referências

- [Scikit-learn — Model Evaluation](#)
- [Scikit-learn — Cross-validation](#)
- [Optuna Documentation](#)
- Géron, A. *Hands-On Machine Learning* — Cap. 2 (End-to-End ML Project), Cap. 3 (Classification)
- Slides da Aula 5 — `Aula5.pptx`
- Notebooks: `aula_ml_classificacao_v2.ipynb` · `Exemplo_aula5_tuning_rand.ipynb`